

BANCO DE DADOS HIGH LOAD

PostgreSQL & Redis: Otimização para 10.000 QPS

Domine particionamento de tabelas, indexação avançada B-Tree/GIN, réplicas de leitura, clusterização Redis e resolução de gargalos de alto tráfego.

[**Nível: Avançado**]

[**Duração: 14h**]

[**10 Módulos**]

[**Tech: PostgreSQL, Redis, Docker**]

#> Módulo 1: Fundamentos de Alto Desempenho

O desenvolvimento de aplicativos modernos requer bancos de dados capazes de lidar com milhares de consultas por segundo (QPS). A diferença entre um desenvolvedor júnior e um Arquiteto de Banco de Dados reside na compreensão do que acontece no hardware quando uma consulta SQL é executada.

O Mito das 10.000 QPS

Alcançar 10k QPS não é sobre usar um 'banco de dados mais rápido', mas eliminar gargalos de I/O (leitura/gravação de disco) e aproveitar a RAM. PostgreSQL pode superar esse limite se soubermos como estruturar dados e lidar com concorrência (ACID).

#> Módulo 2: Internals do PostgreSQL (MVCC & WAL)

Para otimizar o PostgreSQL, devemos entender como ele processa dados internamente.

MVCC (Controle de Concorrência Multiversão)

Ao fazer um UPDATE, o Postgres NÃO exclui a linha antiga. Em vez disso, cria uma nova versão. Isso permite que outras transações continuem lendo a linha antiga sem serem bloqueadas. No entanto, isso gera 'Dead Tuples' (lixo).

Write-Ahead Logging (WAL)

Para ser rápido e seguro contra quedas de energia, o Postgres grava todas as alterações em um arquivo sequencial chamado WAL antes de gravar no arquivo de dados real.

```
-- Visualizar estatísticas de tuplas mortas que precisam de Vacuum:  
SELECT relname, n_dead_tup  
FROM pg_stat_user_tables  
ORDER BY n_dead_tup DESC;
```

O processo AUTOVACUUM é crucial: limpa tuplas mortas. Nunca desative o autovacuum em produção.

#> Módulo 3: Indexação Avançada B-Tree

Um índice não é mágica; é uma estrutura de dados (geralmente uma Árvore Balanceada - B-Tree) que salva ponteiros para a localização física das linhas no disco.

Índices Compostos e Ordem

Se você cria um índice em (A, B), ele só é útil se a consulta buscar por A, ou por A e B. Se buscar apenas por B, o índice composto NÃO funciona (regra do prefixo mais à esquerda).

```
-- Criar um índice composto:  
CREATE INDEX idx_user_status_date ON users (status, created_at DESC);  
  
-- Esta consulta usará o índice perfeitamente:  
SELECT * FROM users WHERE status = 'active' ORDER BY created_at DESC;
```

Index-Only Scans e Fillfactor

Se o seu SELECT pedir apenas colunas que JÁ estão no índice, o Postgres não precisa ler a tabela no disco. Isso é chamado de Index-Only Scan e é o Santo Graal do desempenho de leitura.

#> Módulo 4: Índices Especiais (GIN & GiST)

B-Tree é excelente para igualdades e intervalos (>, <). Mas falha em buscas dentro de arrays, documentos JSON ou texto livre (Full-Text Search).

O Índice GIN (Índice Invertido Generalizado)

O GIN é ideal para colunas JSONB. Ele constrói um 'índice invertido', mapeando cada chave/valor JSON para as linhas que os contêm.

```
-- Assumindo uma coluna JSONB chamada 'metadata'
CREATE INDEX idx_gin_metadata ON products USING GIN (metadata);

-- Busca ultra-rápida dentro de um JSON gigante:
SELECT * FROM products WHERE metadata @> '{"color": "red"}';
```

Sem GIN, o Postgres teria que fazer um 'Seq Scan' (ler toda a tabela) e abrir cada JSON um por um, destruindo o desempenho.

#> Módulo 5: Query Tuning e EXPLAIN ANALYZE

Nunca tente adivinhar por que uma consulta é lenta. Use o profiler interno do PostgreSQL.

Anatomia do EXPLAIN ANALYZE

```
EXPLAIN ANALYZE SELECT * FROM orders WHERE user_id = 15;
```

Ao executar isso, o Postgres lhe dirá:

- Seq Scan: Leu toda a tabela (Perigo se for grande!).
- Index Scan: Usou o índice e depois foi à tabela buscar os dados.
- Execution Time: O tempo real levado em milissegundos.
- Buffers: Quantos blocos de memória teve que tocar.

Se vir um 'Seq Scan' numa tabela enorme, você precisa de um índice ou a condição de busca está ocultando o índice (ex: `WHERE LOWER(email) = ...` requer índice funcional).

#> Módulo 6: Particionamento de Tabelas

Quando uma tabela atinge dezenas de milhões de linhas, até índices ficam lentos pois a B-Tree fica muito profunda. A solução é dividir fisicamente a tabela.

Particionamento Declarativo por Faixa

```
-- 1. Criamos a tabela 'pai' definindo o tipo de partição
CREATE TABLE sales (
    id serial,
    amount numeric,
    created_at date
) PARTITION BY RANGE (created_at);

-- 2. Criamos as partições 'filhas'
CREATE TABLE sales_2024_q1 PARTITION OF sales
    FOR VALUES FROM ('2024-01-01') TO ('2024-04-01');

CREATE TABLE sales_2024_q2 PARTITION OF sales
    FOR VALUES FROM ('2024-04-01') TO ('2024-07-01');
```

Ao fazer um SELECT de Março de 2024, o Postgres ignora completamente a tabela `sales\_2024\_q2`. Você reduziu o volume de busca pela metade instantaneamente.

#> Módulo 7: Replicação e Pooling (PgBouncer)

Nenhum servidor físico suporta leituras infinitas. Para escalar horizontalmente, criamos réplicas.

Read Replicas

Por meio da 'Streaming Replication', o Postgres copia registros WAL da base principal (Master) para um servidor escravo (Replica) em tempo real. Seu app envia os SELECTs (leituras) para a réplica, liberando a principal apenas para INSERTs e UPDATEs.

Connection Pooling com PgBouncer

Postgres consome RAM (~10MB) para cada conexão. Se seu app abre 5000 conexões, o Postgres entra em colapso. PgBouncer fica no meio: recebe as 5000 do app e as reutiliza sobre 100 conexões reais ao Postgres.

#> Módulo 8: Arquitetura In-Memory (Redis)

Mesmo com índices, ler de um SSD leva milissegundos. Para latências de microssegundos, lemos direto da RAM. Aqui entra o Redis.

Estruturas de Dados Nativas

Redis não salva tabelas, salva estruturas: Strings, Hashes, Lists, Sets e Sorted Sets.

```
# Usando redis-cli:
# Salvar uma sessão num Hash
HSET session:123 user_id 45 name "Neo"

# Recuperar o nome em O(1) (microssegundos)
HGET session:123 name
```

#> Módulo 9: Estratégias de Cache Extremo

Quando escrevemos no Redis e quando no Postgres?

Padrão Cache-Aside

1. App pede dados.
2. Busca no Redis primeiro.
3. Cache Hit: devolve imediatamente.
4. Cache Miss: vai no Postgres, processa, devolve e SALVA no Redis com um TTL (Time-To-Live).

```
import redis, psycopg2

r = redis.Redis(host='localhost', port=6379, db=0)

def get_user(user_id):
    # Tentar cache primeiro
    user_data = r.get(f"user:{user_id}")
    if user_data:
        return user_data

    # Se falhar, ir ao Postgres...
    # db_data = run_sql(f"SELECT * FROM users WHERE id={user_id}")

    # Salvar no Redis com expiração de 10 minutos
    r.setex(f"user:{user_id}", 600, db_data)
    return db_data
```

Isso protege o Postgres de consultas repetitivas massivas (Ex: contagem de likes num post viral).

#> Módulo 10: Redis Clustering e Sharding

Se você tiver muito cache, enche a RAM de um único servidor. Redis Cluster distribui as chaves (Sharding) em dezenas de servidores automaticamente.

Resolução de 'Hot Keys'

Se um influenciador posta um vídeo, a chave `video:123:likes` receberá milhões de requests. A solução é adicionar cache local na aplicação (L1 Cache) antes de consultar o Redis (L2), mitigando a avalanche.

Parabéns! Você concluiu o treinamento de High Load. Pronto para desenhar arquiteturas que suportam milhões de usuários diários.

HIGH-PERFORMANCE DATABASE ENGINEER

Certificação concedida a:

[DATABASE ARCHITECT]

Por dominar o design e o dimensionamento de bancos de dados relacionais e em memória,
resolvendo gargalos de concorrência e alcançando alta taxa de I/O em ambientes de
produção massivos.

TIMESTAMP: 2026-08-12 // SYS_DBA_SIGNATURE